

Free Distance and Puncturing

Basic Idea of Convolutional Coding

A convolutional code is a channel-coding method in which the encoder processes a continuous input stream rather than separate fixed blocks. The output bits depend on the current input bit and also on a few previous input bits stored in memory. For example, a rate-1/2 encoder with generators $(7, 5)_8$, where one input bit enters at each step and two coded bits leave. Since the encoder has memory $m = 2$, its state is determined by the two previous input bits, giving four possible states.

The possible state transitions of a convolutional encoder form a trellis. During decoding, the Viterbi algorithm searches this trellis and selects the most likely transmitted path. Therefore, the strength of a convolutional code depends on how far apart competing trellis paths are.

Free Distance

For block codes, error-correcting strength is measured by the minimum Hamming distance $d_{\min}$. For convolutional codes, the corresponding measure is the free distance, d_{free} . It is defined as the smallest output Hamming weight of any nonzero path that leaves the all-zero state and later returns to the all-zero state.

Convolutional codes are linear. The difference between two valid paths is itself a valid path, so comparing two competing paths can be reduced to comparing a nonzero excursion from the all-zero path.

In the $(7, 5)_8$ encoder, the input sequence 100 produces the output sequence 11 10 11. This output has five ones, so its Hamming weight is 5. The source material shows that no smaller nonzero excursion exists; therefore:

$$d_{\text{free}} = 5$$

Free distance determines the correction capability per error event:

$$t = \text{floor}((d_{\text{free}} - 1) / 2)$$

For $d_{\text{free}} = 5$, the decoder can correct up to two coded-bit errors in a minimum-distance error event. If three critical bits are corrupted, the received sequence may become closer to an incorrect trellis path, so Viterbi decoding may choose the wrong message. Thus, free distance is not a total error allowance for the whole message; it describes the separation between competing paths in the trellis.

Puncturing

Puncturing is a technique used to increase the code rate of a convolutional code without changing the basic encoder or the Viterbi trellis. A low-rate code first generates all coded bits, and then selected coded bits are not transmitted according to a fixed repeating pattern known by both transmitter and receiver.

For example, a rate-1/2 encoder sends two coded bits for every input bit. If, over two stages, the encoder produces four coded bits but one of them is deleted, then only three coded bits are transmitted for two input bits. The effective rate becomes:

$$R = 2 / 3$$

More generally, if a puncturing pattern has period p and contains $w(P)$ transmitted positions, then:

$$R = kp / w(P)$$

Puncturing works because the decoder knows exactly which positions were not sent. These deleted positions are treated as carrying no information. In soft-decision decoding, their LLR is set to zero; in hard-decision decoding, they add zero to the branch metric. Therefore, the trellis remains the same, but each branch metric is computed from fewer received bits.

The advantage of puncturing is flexibility: the same encoder can support rates such as 1/2, 2/3, and 3/4. The disadvantage is reduced protection. Since some coded bits are removed, the free distance may decrease. In the example given in the slide, puncturing the rate-1/2 code to rate 2/3 can reduce d_{free} from 5 to 3, so the correction capability falls from two errors per event to one error per event. Thus, puncturing trades error protection for higher throughput.